

Fase 15 — Fragmentación Híbrida

Materia

BDD

Continuación directa de la Fase 14. Los seis nodos del laboratorio tienen sus snapshots `fase14-completa` tomados. En esta fase **no se crean nuevas máquinas virtuales ni se redistribuyen datos**: el objetivo es analizar, demostrar y optimizar la **fragmentación híbrida** como propiedad emergente de la arquitectura construida en las Fases 13 y 14. La combinación de fragmentación horizontal por región (`clientes` , `pedidos` y `detalle_pedidos` repartidos entre nodo04 y nodo05) con fragmentación vertical por grupo de columnas (`v_productos_basico` en nodo04 y `v_productos_detalle` en nodo05) constituye un escenario híbrido real: las consultas del cliente conectado a nodo06 pueden necesitar acceder a uno, dos o los cuatro “bloques de datos” distribuidos —dos horizontales y dos verticales— dependiendo de sus predicados y de qué columnas proyectan. Esta fase construye un catálogo de ocho consultas de localidad creciente (H-1 a H-8), introduce índices de optimización en los shards, demuestra el enrutamiento automático de escrituras y culmina con la creación de objetos de negocio —la vista `reporte_pedidos_detallado` y el procedimiento `consulta_regional()` — que encapsulan la distribución y verifican la transparencia completa ante el cliente final.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Definir formalmente la **fragmentación híbrida inter-tabla** y distinguirla de la fragmentación híbrida intra-tabla teórica, aplicando ambas definiciones al esquema `lab_bdd` del laboratorio.
2. Clasificar consultas según su **grado de localidad distribuida** —cuántos nodos físicos distintos accede Spider para resolverlas— y construir un catálogo de ocho consultas representativas con gradación de menor a mayor distribución.
3. Crear **índices compuestos de optimización** en nodo04 y nodo05 para los patrones de JOIN más comunes, y explicar cómo esos índices mejoran las subconsultas generadas internamente por Spider en cada shard remoto.

4. Demostrar el **empuje de predicados** (*predicate pushdown*) en tablas Spider particionadas (`clientes` , `pedidos`) y explicar por qué la VIEW `detalle_pedidos` (UNION ALL) no se beneficia del mismo mecanismo automáticamente.
 5. Aplicar la **proyección de columnas** como técnica de optimización vertical: elegir `v_productos_basico` en lugar de la VIEW `productos` cuando las columnas TEXT de detalle no son necesarias, evitando el acceso a nodo05.
 6. Reescribir la **consulta híbrida canónica del DDD** (Consulta Distribuida 4, Fase 9) en versiones de menor grado de localidad y demostrar que producen resultados correctos con menor tráfico de red.
 7. Demostrar el **enrutamiento automático de escrituras** (INSERT, UPDATE, DELETE) a través del coordinador Spider hacia el shard correcto, e identificar los casos de *fan-out* (escrituras sin predicado de región).
 8. Construir la vista `reporte_pedidos_detallado` y el procedimiento almacenado `consulta_regional()` que encapsulan toda la complejidad distribuida y exponen una interfaz limpia al cliente de la aplicación.
 9. Verificar la **transparencia completa de distribución** —fragmentación, ubicación y replicación— desde la perspectiva de un usuario final que solo conoce la IP de nodo06.
 10. Cerrar la fase con el snapshot `fase15-completa` en los seis nodos del laboratorio.
-

B. Conceptos teóricos necesarios

1. Fragmentación híbrida: taxonomía y aplicación al laboratorio.

La fragmentación híbrida puede manifestarse de dos formas:

- *Intra-tabla*: la misma relación se fragmenta aplicando tanto una estrategia horizontal como una vertical. Teóricamente, `productos` podría dividirse por rangos de precio (horizontal) y también por grupos de columnas (vertical) dentro del mismo diseño. MariaDB Spider no soporta este esquema en una sola definición de tabla.
- *Inter-tabla*: distintas relaciones del mismo esquema usan estrategias de fragmentación diferentes. Este es el caso del laboratorio: `clientes` , `pedidos` y `detalle_pedidos` están fragmentadas **horizontalmente** por región, mientras que `productos` está fragmentada **verticalmente** por grupo de columnas.

La hibridez emerge de combinar ambas estrategias en el mismo sistema, no en la misma tabla. Una consulta que une `pedidos` (horizontal) con `productos` (vertical) cruza, sin que el cliente lo perciba, los dos tipos de fragmentación simultáneamente.

2. Grado de localidad de una consulta distribuida.

El grado de localidad mide cuántos nodos físicos distintos debe consultar Spider para resolver una consulta. Con dos nodos de sharding (nodo04 y nodo05), los posibles grados son:

Grado	Descripción	Nodos accedidos
0	Un solo nodo (máxima localidad)	Solo nodo04 o solo nodo05
1a	Horizontal pura (sin vertical)	nodo04 + nodo05 para H
1b	Vertical pura (sin horizontal)	nodo04 + nodo05 para V
2	Híbrida total (H + V)	nodo04 + nodo05 para H y para V

El grado 0 requiere dos condiciones simultáneas: filtro de región que cubra solo un shard (por ejemplo, `region IN ('norte','este')`) y no necesitar las columnas TEXT de `v_productos_detalle` (es decir, usar `v_productos_basico` directamente).

3. Descomposición interna de consultas por el motor Spider.

Spider traduce cada `SELECT` del coordinador en subconsultas dirigidas a los shards remotos:

- Para **tablas Spider particionadas** (`clientes`, `pedidos`): evalúa el `WHERE` para determinar qué particiones son relevantes, genera una subconsulta por partición y recombina los resultados mediante UNION ALL implícito.
- Para **tablas Spider simples** (`v_productos_basico`, `v_productos_detalle`, `spider_detalle_nodo04`, `spider_detalle_nodo05`): genera exactamente una subconsulta dirigida al nodo configurado en el `COMMENT` del `CREATE TABLE`.
- Para **VIEWS** (`detalle_pedidos`, `productos`): primero el optimizador de MariaDB expande la VIEW, luego Spider procesa cada tabla Spider subyacente de forma independiente.

4. Empuje de predicados y sus límites en Spider VIEWS.

El *predicate pushdown* ocurre cuando Spider puede incluir en la subconsulta remota el predicado de la consulta exterior, reduciendo las filas transferidas por la red:

- **Funciona en tablas Spider particionadas:** `WHERE region = 'norte'` sobre `clientes` → Spider envía `SELECT ... WHERE region = 'norte'` directamente a frag_A en nodo04.
- **Funciona en tablas Spider simples:** `WHERE id = 5` sobre `v_productos_basico` → la subconsulta a nodo04 incluye `WHERE id = 5`.
- **No funciona automáticamente para la VIEW `detalle_pedidos`**: al ser una VIEW definida como `UNION ALL (spider_detalle_nodo04, spider_detalle_nodo05)`, un predicado como `WHERE pedido_id IN (...)` que viene de un JOIN con `pedidos` no se empuja dentro del UNION ALL. Spider descarga todas las filas de ambas tablas subyacentes y el coordinador filtra localmente. La optimización manual consiste en reemplazar la VIEW por la tabla Spider directa (`spider_detalle_nodo04` o `spider_detalle_nodo05`) cuando se sabe qué shard contiene los datos relevantes.

5. Proyección de columnas como técnica de optimización vertical.

Para la fragmentación vertical, las columnas proyectadas en el `SELECT` determinan qué nodo(s) accede Spider:

- `SELECT sku, nombre, precio FROM v_productos_basico` → solo nodo04 (nunca toca nodo05).
- `SELECT * FROM productos` (VIEW JOIN) → nodo04 (basico) y nodo05 (detalle).

Principio: si la consulta no necesita las columnas TEXT de detalle (`descripcion`, `ficha_tecnica`, `imagen_url`, `peso_kg`), usar `v_productos_basico` directamente evita completamente el acceso a `v_productos_detalle` en nodo05. Esta es la optimización vertical más impactante disponible en el diseño actual.

6. Índices en nodos remotos y su impacto en Spider.

Spider envía la subconsulta al shard remoto y ese nodo la ejecuta usando su motor InnoDB local. Los índices en el shard afectan directamente la eficiencia de la subconsulta:

- Sin índice en la columna de JOIN: el shard hace un full scan local antes de devolver las filas al coordinador.
- Con índice compuesto (`region`, `cliente_id`) en `pedidos`: el shard puede resolver el JOIN con `clientes` usando acceso por índice, reduciendo las filas procesadas localmente.

La creación de índices en los shards es completamente transparente para el cliente en nodo06; solo afecta al plan de ejecución interno de cada shard.

7. Enrutamiento de escrituras a través del coordinador Spider.

Para operaciones de escritura, Spider aplica la misma lógica de particionamiento:

- **INSERT:** Spider evalúa el valor de la columna de partición (`region`) en la fila a insertar y dirige el INSERT al shard correspondiente. Un INSERT sin valor de región producirá un error, ya que Spider no puede determinar la partición destino.
- **UPDATE/DELETE con predicado de región:** Spider poda y envía la operación únicamente al shard relevante (*targeted write*). Ejemplo: `UPDATE ... WHERE region = 'norte'` → solo nodo04.
- **UPDATE/DELETE sin predicado de región (*fan-out*):** Spider puede necesitar buscar la fila en ambos shards. Primero localiza en cuál está la fila, luego ejecuta la modificación. Este patrón genera el doble de tráfico de red y debe evitarse en consultas frecuentes.
- **INSERT/UPDATE en tablas Spider simples (`v_productos_basico` , `v_productos_detalle`):** siempre se dirigen al nodo configurado en el `COMMENT` ; no hay particionamiento.

8. Transparencia de distribución completa.

Al finalizar la Fase 15, el laboratorio implementa tres de los cuatro tipos de transparencia definidos en la Fase 0:

Tipo	Mecanismo	Estado
Fragmentación	El cliente consulta <code>clientes</code> , <code>productos</code> , etc. sin saber que los datos están divididos	Implementada desde Fase 14
Ubicación	El cliente siempre conecta a <code>192.168.56.106</code> ; ignora nodo04 y nodo05	Implementada desde Fase 14
Replicación	nodo01–03 replican <code>lab_bdd</code> entre sí de forma invisible para el cliente	Implementada desde Fase 10–11
Concurrencia	Control de versiones concurrentes; pruebas de aislamiento	Se aborda en Fase 17

C. Prerrequisitos

Estado requerido de las máquinas virtuales

Nodo	Snapshot	Estado funcional esperado
<code>bdd-nodo04</code>	<code>fase14-completa</code>	SHARD-A: <code>clientes</code> (10, norte+este), <code>pedidos</code> (10), <code>detalle_pedido</code> <code>s</code> (~18), <code>v_productos_ba</code> <code>sico</code> (10 filas); <code>bind-</code> <code>address=0.0.0.</code> <code>0</code> ; <code>server_id=4</code>
<code>bdd-nodo05</code>	<code>fase14-completa</code>	SHARD-B: <code>clientes</code> (10, sur+oeste), <code>pedidos</code> (10), <code>detalle_pedido</code> <code>s</code> (~18), <code>v_productos_de</code> <code>talle</code> (10 filas); <code>bind-</code> <code>address=0.0.0.</code> <code>0</code> ; <code>server_id=5</code>
<code>bdd-nodo06</code>	<code>fase14-completa</code>	COORDINADOR: Spider <code>ACTIVE</code> ; <code>lab_bdd</code> con tablas Spider (<code>clientes</code> , <code>pedidos</code>) PARTITION LIST region; <code>spider_detalle</code> <code>_nodo04/05</code> ; <code>v_productos_ba</code> <code>sico/detalle</code>

```
simples) y  
VIEWS  
( detalle_pedido  
os UNION ALL;  
productos  
JOIN);  
server_id=6;  
log_bin=OFF
```

`bdd-nodo01`, `bdd-nodo02` y `bdd-nodo03` permanecen **apagados** durante toda esta fase.
No son necesarios para los ejercicios de fragmentación híbrida.

Conocimiento técnico requerido

- Particionamiento `LIST COLUMNS` y poda de particiones (Fase 12).
- Fragmentación horizontal con predicados `frag_A` / `frag_B` y fragmentación derivada de `detalle_pedidos` (Fase 13).
- Motor Spider, `CREATE SERVER`, tablas Spider particionadas y simples, VIEWS de reconstrucción (Fase 14).
- El Documento de Diseño Distribuido `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md`, especialmente la Consulta Distribuida 4 de la sección D.5, que es la consulta canónica híbrida que se analiza en profundidad en esta fase.

D. Procedimiento paso a paso

Paso 1 — Iniciar nodo04, nodo05 y nodo06; verificar el estado heredado de la Fase 14 (conteos de filas, `bind-address`, `server_id`, Spider `ACTIVE`, tablas y VIEWS existentes).

Paso 2 — Revisar los índices actuales en nodo04 y nodo05 con `information_schema.STATISTICS`.

Paso 3 — Crear índices compuestos de optimización en nodo04 y nodo05 para los patrones de JOIN más comunes en consultas híbridas.

Paso 4 — Ejecutar el catálogo de ocho consultas híbridas desde nodo06 (H-1 a H-8), organizadas de menor a mayor grado de distribución.

Paso 5 — Analizar con `EXPLAIN` el impacto del predicate pushdown y la proyección de columnas en las particiones accedidas; comparar versiones optimizadas vs. no optimizadas.

Paso 6 — Demostrar operaciones de escritura (INSERT, UPDATE, DELETE) a través del coordinador Spider, con verificación directa en los shards.

Paso 7 — Crear la vista de negocio `reporte_pedidos_detallado` y el procedimiento almacenado `consulta_regional()` en nodo06.

Paso 8 — Demostrar transparencia total: crear el usuario `app_final` en nodo06 y ejecutar consultas como si fuera una aplicación cliente sin conocimiento de la distribución.

Paso 9 — Limpiar los datos de prueba insertados en el Paso 6.

Paso 10 — Apagar los tres nodos activos y tomar el snapshot `fase15-completa` en los seis nodos del laboratorio.

E. Comandos completos

*Los bloques **(host)** se ejecutan en PowerShell en Windows. Los bloques **(VM — nodoXX)** se ejecutan en una sesión SSH al nodo indicado. Los bloques SQL dentro de `sudo mariadb` se ejecutan en el prompt del motor MariaDB.*

E.1 Iniciar nodo04, nodo05 y nodo06; verificar el estado de la Fase 14 (host)

 PowerShell

```
VBoxManage startvm "bdd-nodo04" --type headless
VBoxManage startvm "bdd-nodo05" --type headless
VBoxManage startvm "bdd-nodo06" --type headless
Start-Sleep -Seconds 35
```

Abrir tres sesiones SSH simultáneas:

PowerShell

```
ssh bddadmin@192.168.56.104 # Terminal 1 - nodo04
ssh bddadmin@192.168.56.105 # Terminal 2 - nodo05
ssh bddadmin@192.168.56.106 # Terminal 3 - nodo06
```

Verificar el estado en **nodo04**:

Bash

```
sudo mariadb lab_bdd << 'EOF'
SELECT 'clientes'          AS tabla, COUNT(*) AS filas,
GROUP_CONCAT(DISTINCT region ORDER BY region) AS regiones
FROM   clientes
UNION ALL
SELECT 'pedidos',          COUNT(*), GROUP_CONCAT(DISTINCT region ORDER
BY region)
FROM   pedidos
UNION ALL
SELECT 'detalle_pedidos',  COUNT(*), NULL
FROM   detalle_pedidos
UNION ALL
SELECT 'v_productos_basico', COUNT(*), NULL
FROM   v_productos_basico;
SHOW VARIABLES LIKE 'server_id';
SHOW VARIABLES LIKE 'bind_address';
EOF
```

Salida esperada: 10 / 10 / ~18 / 10 filas; regiones solo `este,norte`; `server_id = 4`;
`bind_address = 0.0.0.0`.

Verificar en **nodo05**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
SELECT 'clientes'           AS tabla, COUNT(*) AS filas,
GROUP_CONCAT(DISTINCT region ORDER BY region) AS regiones
FROM   clientes
UNION ALL
SELECT 'pedidos',           COUNT(*), GROUP_CONCAT(DISTINCT region ORDER
BY region)
FROM   pedidos
UNION ALL
SELECT 'detalle_pedidos',   COUNT(*), NULL
FROM   detalle_pedidos
UNION ALL
SELECT 'v_productos_detalle', COUNT(*), NULL
FROM   v_productos_detalle;
SHOW VARIABLES LIKE 'server_id';
SHOW VARIABLES LIKE 'bind_address';
EOF
```

Salida esperada: 10 / 10 / ~18 / 10 filas; regiones solo `oeste,sur`; `server_id = 5` .

Verificar el coordinador en **nodo06**:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Comprobar Spider activo
SELECT PLUGIN_NAME, PLUGIN_STATUS FROM information_schema.PLUGINS
WHERE  PLUGIN_NAME = 'SPIDER';
-- Verificar objetos del esquema
SELECT TABLE_NAME AS objeto, TABLE_TYPE AS tipo, ENGINE
FROM  information_schema.TABLES
WHERE  TABLE_SCHEMA = 'lab_bdd'
ORDER BY TABLE_TYPE DESC, TABLE_NAME;
-- Contar filas globales desde el coordinador
SELECT 'clientes (Spider PARTITION)' AS fuente, COUNT(*) AS filas
FROM clientes
UNION ALL
SELECT 'pedidos (Spider PARTITION)', COUNT(*)
FROM pedidos
UNION ALL
SELECT 'detalle_pedidos (VIEW UNION ALL)', COUNT(*)
FROM detalle_pedidos
UNION ALL
SELECT 'v_productos_basico (Spider-nodo04)', COUNT(*)
FROM v_productos_basico
UNION ALL
SELECT 'v_productos_detalle (Spider-nodo05)', COUNT(*)
FROM v_productos_detalle
UNION ALL
SELECT 'productos (VIEW JOIN distribuido)', COUNT(*)
FROM productos;
EOF
```

Salida esperada: Spider **ACTIVE** ; 6 tablas + 2 VIEWS; conteos 20 / 20 / ~36 / 10 / 10 / 10.

E.2 Revisar índices actuales y crear índices de optimización en los shards

E.2.1 Análisis de índices existentes (VM — nodo04)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Auditoría de índices actuales en todas las tablas del shard
SELECT TABLE_NAME      AS tabla,
       INDEX_NAME        AS indice,
       SEQ_IN_INDEX      AS posicion,
       COLUMN_NAME       AS columna,
       NON_UNIQUE        AS no_unico,
       NULLABLE          AS nutable
FROM   information_schema.STATISTICS
WHERE  TABLE_SCHEMA = 'lab_bdd'
ORDER BY TABLE_NAME, INDEX_NAME, SEQ_IN_INDEX;
EOF
```

Los índices heredados de las Fases 13 y 14 son:

Tabla	Índice	Columnas
clientes	PRIMARY	id
clientes	uk_email	email
clientes	idx_region	region
pedidos	PRIMARY	id
pedidos	idx_cliente	cliente_id
pedidos	idx_region	region
detalle_pedido s	PRIMARY	id
detalle_pedido s	idx_pedido	pedido_id
detalle_pedido s	idx_producto	producto_id
v_productos_ba sico	PRIMARY	id
v_productos_ba sico	uq_sku	sku

E.2.2 Crear índices compuestos de optimización en nodo04

Los nuevos índices cubren los patrones de JOIN más comunes en consultas híbridas:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- — ÍNDICE 1 —————
-- Tabla: pedidos
-- Columnas: (region, cliente_id)
-- Patrón cubierto: consultas que filtran por región Y hacen JOIN a clientes.
-- Sin este índice: el shard usa idx_region para la poda de región pero luego
-- hace un full scan del fragmento para resolver el JOIN.
ALTER TABLE pedidos
  ADD INDEX IF NOT EXISTS idx_region_cliente (region, cliente_id);
-- — ÍNDICE 2 —————
-- Tabla: clientes
-- Columnas: (region, id)
-- Patrón cubierto: consultas WHERE region IN (...) que necesitan id
-- para el JOIN con pedidos. Permite index-only scan en muchos casos.
ALTER TABLE clientes
  ADD INDEX IF NOT EXISTS idx_region_id (region, id);
-- — ÍNDICE 3 —————
-- Tabla: v_productos_basico
-- Columnas: (categoria)
-- Patrón cubierto: consultas GROUP BY o WHERE por categoría de producto
-- (frecuentes en reportes de ventas por línea de negocio).
ALTER TABLE v_productos_basico
  ADD INDEX IF NOT EXISTS idx_categoria (categoria);
-- — ÍNDICE 4 —————
-- Tabla: v_productos_basico
-- Columnas: (precio)
-- Patrón cubierto: filtros de rango por precio (búsquedas con
-- BETWEEN o comparación de precio en el catálogo de productos).
ALTER TABLE v_productos_basico
  ADD INDEX IF NOT EXISTS idx_precio (precio);
-- Verificar los nuevos índices
SELECT TABLE_NAME,
       INDEX_NAME,
       GROUP_CONCAT(COLUMN_NAME ORDER BY SEQ_IN_INDEX) AS columnas,
       CASE WHEN NON_UNIQUE = 0 THEN 'ÚNICO' ELSE 'normal' END AS tipo
FROM   information_schema.STATISTICS
WHERE  TABLE_SCHEMA = 'lab_bdd'
GROUP BY TABLE_NAME, INDEX_NAME
ORDER BY TABLE_NAME, INDEX_NAME;
EOF
```

E.2.3 Crear los mismos índices en nodo05

 Bash

```
# En nodo05 – mismos índices para las tablas horizontales equivalentes
sudo mariadb lab_bdd << 'EOF'
ALTER TABLE pedidos
  ADD INDEX IF NOT EXISTS idx_region_cliente (region, cliente_id);
ALTER TABLE clientes
  ADD INDEX IF NOT EXISTS idx_region_id (region, id);
-- v_productos_detalle no requiere índices de categoria/precio porque
-- no contiene esas columnas; sus búsquedas son por id o sku.
-- Verificar
SELECT TABLE_NAME,
       INDEX_NAME,
       GROUP_CONCAT(COLUMN_NAME ORDER BY SEQ_IN_INDEX) AS columnas
FROM   information_schema.STATISTICS
WHERE  TABLE_SCHEMA = 'lab_bdd'
GROUP BY TABLE_NAME, INDEX_NAME
ORDER BY TABLE_NAME, INDEX_NAME;
EOF
```

E.3 Catálogo de consultas híbridas desde nodo06 (H-1 a H-8)

Ejecutar el bloque completo en **nodo06**. Cada consulta incluye un encabezado que describe los fragmentos accedidos y el grado de localidad.

 Bash

```
sudo mariadb lab_bdd << 'EOF'

-- =====
-- CATÁLOGO DE CONSULTAS HÍBRIDAS – Grado de localidad 0 a 2
-- =====
-- -----
-- H-1: GRADO 0 – Localidad máxima (solo nodo04)
--
-- Fragmentos accedidos:
--   · clientes_frag_A      (nodo04) – poda por WHERE region
IN ('norte', 'este')
--   · pedidos_frag_A      (nodo04) – poda por partición frag_A
--   · spider_detalle_nodo04 (nodo04) – tabla Spider directa (NO la VIEW)
--   · v_productos_basico   (nodo04) – Spider simple, siempre en nodo04
--
-- Clave de optimización: se usa spider_detalle_nodo04 DIRECTAMENTE
-- para evitar que la VIEW detalle_pedidos (UNION ALL) acceda a nodo05.
-- -----
SELECT '=== H-1: Grado 0 – Solo nodo04 (máxima localidad) ===' AS consulta;
SELECT c.region,
       CONCAT(c.nombre, ' ', c.apellido) AS cliente,
       c.ciudad,
       p.id                               AS pedido_id,
       p.estado,
       vb.sku,
       vb.nombre                          AS producto,
       vb.categoria,
       vb.precio,
       dp.cantidad,
       dp.subtotal
FROM   clientes      c
JOIN   pedidos        p  ON  p.cliente_id = c.id
JOIN   spider_detalle_nodo04 dp ON dp.pedido_id = p.id
JOIN   v_productos_basico vb ON vb.id       = dp.producto_id
WHERE  c.region IN ('norte', 'este')
ORDER BY c.region, p.id;

-- -----
-- H-2: GRADO 1a – Fragmentación horizontal pura (ambos shards)
--
-- Fragmentos accedidos:
--   · clientes frag_A (nodo04) + frag_B (nodo05) – sin filtro de región
--   · pedidos frag_A (nodo04) + frag_B (nodo05)
-- Sin join a productos: no hay fragmentación vertical involucrada.
-- -----
```

```

SELECT '=== H-2: Grado 1a - Horizontal puro (nodo04+nodo05) ===' AS consulta;
SELECT c.region,
       COUNT(DISTINCT c.id)    AS clientes,
       COUNT(DISTINCT p.id)    AS pedidos,
       ROUND(SUM(p.total), 2)  AS facturacion_total,
       MIN(p.fecha_pedido)    AS primer_pedido,
       MAX(p.fecha_pedido)    AS ultimo_pedido
FROM   clientes c
JOIN   pedidos  p ON p.cliente_id = c.id
GROUP BY c.region
ORDER BY facturacion_total DESC;

--
-- H-3: GRADO 1b - Fragmentación vertical pura (ambos shards)
--
-- Fragmentos accedidos:
--   · v_productos_basico  (nodo04) - columnas operacionales
--   · v_productos_detalle (nodo05) - columnas de detalle TEXT
-- Spider hace JOIN distribuido: basico(nodo04) ⋈ detalle(nodo05)
-- No interviene ninguna tabla de clientes/pedidos.
--
SELECT '=== H-3: Grado 1b - Vertical puro, JOIN distribuido nodo04+nodo05
=== ' AS consulta;
SELECT pr.id,
       pr.sku,
       pr.nombre,
       pr.categoria,
       pr.precio,
       pr.stock,
       LEFT(pr.descripcion, 70) AS descripcion_preview,
       LEFT(pr.ficha_tecnica, 70) AS ficha_preview,
       pr.peso_kg
FROM   productos pr
ORDER BY pr.categoria, pr.nombre;

--
-- H-4: GRADO 2 PARCIAL - Un shard horizontal + ambos verticales
--
-- Fragmentos accedidos:
--   · clientes_frag_A      (nodo04) - poda por WHERE region
IN ('norte', 'este')
--   · pedidos_frag_A      (nodo04) - poda
--   · detalle_pedidos VIEW (nodo04+nodo05) - UNION ALL, sin poda automática
--   · v_productos_basico  (nodo04) - columnas operacionales
--   · v_productos_detalle (nodo05) - columnas TEXT (via VIEW productos)
-- Nodos físicos: nodo04 (horizontal+basico) + nodo05 (detalle+vertical)
--

```

```

SELECT '=== H-4: Grado 2 parcial – Un shard horizontal + ambos verticales
=== ' AS consulta;
SELECT c.region,
       CONCAT(c.nombre, ' ', c.apellido) AS cliente,
       p.id                             AS pedido_id,
       pr.nombre                         AS producto,
       pr.categoria,
       pr.precio,
       LEFT(pr.descripcion, 60)          AS descripcion_preview,
       pr.peso_kg,
       dp.cantidad,
       dp.subtotal
FROM   clientes      c
JOIN   pedidos       p ON p.cliente_id = c.id
JOIN   detalle_pedidos dp ON dp.pedido_id = p.id
JOIN   productos     pr ON pr.id       = dp.producto_id
WHERE  c.region IN ('norte', 'este')
ORDER BY p.id;

-- =====
-- H-5: GRADO 2 PARCIAL – Ambos shards horizontales + un shard vertical
--
-- Fragmentos accedidos:
--   · clientes (frag_A nodo04 + frag_B nodo05) – sin filtro de región
--   · pedidos (frag_A nodo04 + frag_B nodo05)
--   · detalle_pedidos VIEW (nodo04+nodo05)
--   · v_productos_basico (nodo04 únicamente – NO la VIEW productos)
-- Clave: al elegir v_productos_basico en lugar de productos,
-- se evita el acceso adicional a v_productos_detalle en nodo05.
-- =====

SELECT '=== H-5: Grado 2 parcial – Ambos horizontales + solo basico de nodo04
=== ' AS consulta;
SELECT c.region,
       vb.categoria,
       COUNT(DISTINCT c.id) AS clientes_activos,
       COUNT(DISTINCT p.id) AS pedidos,
       SUM(dp.cantidad)     AS unidades_vendidas,
       ROUND(SUM(dp.subtotal), 2) AS facturacion
FROM   clientes      c
JOIN   pedidos       p ON p.cliente_id = c.id
JOIN   detalle_pedidos dp ON dp.pedido_id = p.id
JOIN   v_productos_basico vb ON vb.id    = dp.producto_id
GROUP BY c.region, vb.categoria
ORDER BY c.region, facturacion DESC;

-- =====
-- H-6: GRADO 2 TOTAL – Distribución completa

```

```

--      Consulta Distribuida 4 del DDD (Fase 9, sección D.5)
--
-- Fragmentos accedidos: TODOS
--      · clientes (frag_A + frag_B)
--      · pedidos (frag_A + frag_B)
--      · detalle_pedidos VIEW (nodo04 + nodo05)
--      · v_productos_basico (nodo04)
--      [categoría está en basico → no se necesita v_productos_detalle]
--
-- Esta es la consulta canónica de fragmentación híbrida del laboratorio.
-- Fue verificada localmente en la Fase 9 y a través de Spider en la Fase 14.
--
SELECT '=== H-6: Grado 2 total – Consulta DDD Fase 9 (distribución completa)
=== ' AS consulta;
SELECT c.region,
      COUNT(DISTINCT c.id) AS
clientes_activos,
      COUNT(DISTINCT p.id) AS
pedidos,
      GROUP_CONCAT(DISTINCT vb.categoria
                    ORDER BY vb.categoria SEPARATOR ', ')
AS categorias_compradas,
      ROUND(SUM(dp.subtotal), 2) AS
facturacion_total
FROM   clientes      c
JOIN   pedidos        p ON p.cliente_id = c.id
JOIN   detalle_pedidos dp ON dp.pedido_id = p.id
JOIN   v_productos_basico vb ON vb.id = dp.producto_id
GROUP BY c.region
ORDER BY facturacion_total DESC;
-- Resultado esperado: 4 filas, una por región, con datos de facturación
no nulos
--
-- H-7: GRADO 2 TOTAL – Reporte analítico híbrido avanzado
--
-- Fragmentos accedidos: TODOS, incluyendo v_productos_detalle en nodo05
-- porque peso_kg es una columna TEXT/DECIMAL de v_productos_detalle.
-- Es la consulta de mayor complejidad de distribución del laboratorio:
-- requiere 4 bloques de datos, todos simultáneamente.
--
SELECT '=== H-7: Grado 2 total – Reporte analítico avanzado con ficha de
producto === ' AS consulta;
SELECT vb.nombre AS producto,
      vb.categoria,
      vb.precio AS precio_catalogo,

```

```

        vd.peso_kg,
        LEFT(vd.descripcion, 80)          AS descripcion_corta,
        COUNT(DISTINCT p.region)          AS regiones_de_venta,
        GROUP_CONCAT(DISTINCT p.region
                     ORDER BY p.region
                     SEPARATOR ', ')      AS lista_regiones,
        SUM(dp.cantidad)                  AS total_unidades,
        ROUND(SUM(dp.subtotal), 2)        AS total_facturado,
        ROUND(AVG(dp.precio_unitario), 2) AS precio_promedio_venta
FROM   v_productos_basico vb
JOIN   v_productos_detalle vd ON vd.id      = vb.id
JOIN   detalle_pedidos      dp ON dp.producto_id = vb.id
JOIN   pedidos              p  ON p.id        = dp.pedido_id
GROUP  BY vb.id, vb.nombre, vb.categoria, vb.precio,
vd.peso_kg, vd.descripcion
ORDER  BY total_facturado DESC;
-- -----
-- H-8: GRADO 0 OPTIMIZADO – Versión single-shard de H-6
--
-- Mismo patrón de negocio que H-6 (resumen por región) pero
-- restringido a frag_A (norte+este) y usando spider_detalle_nodo04
-- DIRECTAMENTE (no la VIEW UNION ALL) para evitar el acceso a nodo05.
-- Toda la consulta corre en nodo04 únicamente.
-- -----
SELECT '=== H-8: Grado 0 optimizado – Equivalente regional de H-6 (solo
nodo04) ===' AS consulta;
SELECT c.region,
       COUNT(DISTINCT c.id)          AS
clientes_activos,
       COUNT(DISTINCT p.id)          AS
pedidos,
       GROUP_CONCAT(DISTINCT vb.categoria
                     ORDER BY vb.categoria SEPARATOR ', ')
AS categorias_compradas,
       ROUND(SUM(dp.subtotal), 2)    AS
facturacion_total
FROM   clientes          c
JOIN   pedidos            p  ON p.cliente_id = c.id
JOIN   spider_detalle_nodo04 dp ON dp.pedido_id = p.id
JOIN   v_productos_basico vb ON vb.id        = dp.producto_id
WHERE  c.region IN ('norte', 'este')
GROUP  BY c.region
ORDER  BY facturacion_total DESC;
-- Resultado esperado: 2 filas (norte, este) – subconjunto correcto de H-6

```

```
-- Verificar que el resultado de H-8 coincide con las filas norte+este de H-6  
EOF
```

E.4 Estrategias de optimización: análisis con EXPLAIN (VM — nodo06)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- ANÁLISIS DE PLANES DE EJECUCIÓN CON EXPLAIN
-- Objetivo: demostrar el efecto del predicate pushdown y la
-- proyección de columnas en las particiones accedidas por Spider
-- =====
-- — EXPERIMENTO A: Poda en tablas Spider PARTICIONADAS —————
-- Sin filtro → Spider consulta ambas particiones (frag_A + frag_B)
SELECT '-- A1: clientes SIN filtro — ambas particiones --' AS experimento;
EXPLAIN SELECT id, nombre, region FROM clientes;
-- Con filtro de una sola región → poda a frag_A (nodo04)
SELECT '-- A2: clientes CON region=norte — solo frag_A --' AS experimento;
EXPLAIN SELECT id, nombre, region FROM clientes WHERE region = 'norte';
-- Con IN sobre ambas regiones de un mismo shard → poda a frag_A
SELECT '-- A3: clientes region IN norte,este — solo frag_A --'
AS experimento;
EXPLAIN SELECT id, nombre, region FROM clientes WHERE region IN
('norte', 'este');
-- Con IN mezclando regiones de distintos shards → ambas particiones
SELECT '-- A4: clientes region IN norte,sur — frag_A + frag_B --'
AS experimento;
EXPLAIN SELECT id, nombre, region FROM clientes WHERE region IN
('norte', 'sur');
-- — EXPERIMENTO B: Poda en pedidos (misma lógica) —————
SELECT '-- B1: pedidos con region=sur — solo frag_B (nodo05) --'
AS experimento;
EXPLAIN SELECT id, total FROM pedidos WHERE region IN ('sur', 'oeste');
SELECT '-- B2: pedidos sin filtro — ambas particiones --' AS experimento;
EXPLAIN SELECT COUNT(*), SUM(total) FROM pedidos;
-- — EXPERIMENTO C: VIEW detalle_pedidos — SIN poda automática
-- La VIEW UNION ALL no hace poda aunque el JOIN con pedidos
-- restrinja efectivamente a un shard. Spider descarga ambas tablas
-- subyacentes y el coordinador filtra por pedido_id localmente.
SELECT '-- C1: detalle_pedidos VIEW — siempre accede a ambas tablas Spider --'
AS experimento;
EXPLAIN SELECT COUNT(*) FROM detalle_pedidos;
-- Contraste: tablas Spider directas sí acceden a un solo nodo
SELECT '-- C2: spider_detalle_nodo04 — solo nodo04 --' AS experimento;
EXPLAIN SELECT COUNT(*) FROM spider_detalle_nodo04;
SELECT '-- C3: spider_detalle_nodo05 — solo nodo05 --' AS experimento;
EXPLAIN SELECT COUNT(*) FROM spider_detalle_nodo05;
```

```

-- — EXPERIMENTO D: Proyección de columnas (vertical) —————
-- VIEW productos = JOIN(basico, detalle): siempre accede a ambos nodos
SELECT '-- D1: SELECT * FROM productos — JOIN distribuido (nodo04+nodo05) --'
AS experimento;
EXPLAIN SELECT id, nombre, descripcion FROM productos WHERE id = 1;
-- v_productos_basico: solo nodo04, independientemente del filtro
SELECT '-- D2: SELECT FROM v_productos_basico — solo nodo04 --'
AS experimento;
EXPLAIN SELECT id, nombre, precio FROM v_productos_basico WHERE id = 1;
-- — EXPERIMENTO E: Poda transitiva en JOINS —————
-- ¿Al filtrar clientes por región, se poda también pedidos?
SELECT '-- E1: JOIN clientes+pedidos con region=norte (poda en ambas tablas)
--' AS experimento;
EXPLAIN
SELECT c.nombre, p.total
FROM   clientes c
JOIN   pedidos  p ON p.cliente_id = c.id
WHERE  c.region = 'norte';
-- — RESUMEN VISUAL DE PARTICIONES ACCEDIDAS —————
SELECT '== RESUMEN: qué particiones accede cada elemento ==' AS resumen;
SELECT 'clientes WHERE region=norte'   AS elemento, 'frag_A (nodo04)'
AS particiones
UNION ALL
SELECT 'clientes sin filtro',           'frag_A + frag_B'
UNION ALL
SELECT 'pedidos WHERE region IN(sur,oeste)', 'frag_B (nodo05)'
UNION ALL
SELECT 'detalle_pedidos (VIEW UNION ALL)',  'siempre nodo04
+ nodo05'
UNION ALL
SELECT 'spider_detalle_nodo04 (tabla directa)', 'solo nodo04'
UNION ALL
SELECT 'v_productos_basico (Spider simple)',  'solo nodo04'
UNION ALL
SELECT 'v_productos_detalle (Spider simple)', 'solo nodo05'
UNION ALL
SELECT 'productos (VIEW JOIN basico+detalle)', 'nodo04 + nodo05';
EOF

```

E.5 Operaciones de escritura a través del coordinador Spider (VM — nodo06)

 Bash

```
sudo mariadb lab_bdd << 'EOF'

-- =====
-- ENRUTAMIENTO AUTOMÁTICO DE ESCRITURAS
-- =====
-- — INSERT con region='norte' → Spider enruta a frag_A (nodo04) —
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Escritura', 'FragA', 'escritura.fraga@lab.test',
        '5500000099', 'norte', 'Monterrey');
SELECT 'Verificar INSERT region=norte desde nodo06' AS paso,
       id, nombre, apellido, region, ciudad
FROM   clientes
WHERE  email = 'escritura.fraga@lab.test';
-- — INSERT con region='sur' → Spider enruta a frag_B (nodo05) —
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Escritura', 'FragB', 'escritura.fragb@lab.test',
        '5500000098', 'sur', 'Guadalajara');
SELECT 'Verificar INSERT region=sur desde nodo06' AS paso,
       id, nombre, apellido, region, ciudad
FROM   clientes
WHERE  email = 'escritura.fragb@lab.test';
-- — Conteo post-inserción por región —————
SELECT region, COUNT(*) AS clientes FROM clientes GROUP BY region ORDER
BY region;
-- Esperado: norte=6, sur=6, este=5, oeste=5 (los shards originales tenían
-- distribución 5 por región; los INSERTs añaden 1 a norte y 1 a sur)
-- — INSERT en tabla Spider simple → siempre a nodo04 —————
INSERT INTO v_productos_basico
       (id, sku, nombre, categoria, precio, stock, fecha_creacion)
VALUES (11, 'SKU-PRUEBA-011', 'Producto de Prueba Fase15',
        'Prueba', 99.99, 5, NOW());
SELECT 'Verificar INSERT en v_productos_basico (nodo04)' AS paso,
       id, sku, nombre, categoria, precio, stock
FROM   v_productos_basico
WHERE  id = 11;
-- — UPDATE con predicado de región (escritura dirigida) —————
-- Spider poda a frag_A (nodo04) porque region = 'norte'
UPDATE clientes
SET    ciudad = 'San Pedro Garza García'
WHERE  email = 'escritura.fraga@lab.test'
```

```

    AND region = 'norte';
SELECT 'Verificar UPDATE dirigido (region=norte → solo nodo04)' AS paso,
      id, nombre, ciudad, region
FROM   clientes
WHERE  email = 'escritura.fraga@lab.test';
-- — UPDATE sin predicado de región (fan-out) —————
-- Spider puede necesitar localizar la fila en ambos shards
-- antes de ejecutar la actualización. Patrón a evitar en consultas
-- de alta frecuencia.
UPDATE clientes
SET    telefono = '55000000097'
WHERE  email = 'escritura.fragb@lab.test';
SELECT 'Verificar UPDATE fan-out (sin predicado region)' AS paso,
      id, nombre, telefono, region
FROM   clientes
WHERE  email = 'escritura.fragb@lab.test';
-- — DELETE con predicado de región (eliminación dirigida) ———
-- Solo accede a nodo04 (region='norte')
DELETE FROM clientes
WHERE  email = 'escritura.fraga@lab.test'
      AND region = 'norte';
SELECT 'Verificar DELETE dirigido (debe ser 0 filas)' AS paso,
      COUNT(*) AS debe_ser_cero
FROM   clientes
WHERE  email = 'escritura.fraga@lab.test';
EOF

```

Verificar desde los shards que los datos se distribuyeron correctamente:

 Bash

```
# Verificar que fragb está en nodo05
mysql -h 192.168.56.105 \
-u spider_user \
-p'Spider_2025!' \
lab_bdd \
-e "SELECT 'nodo05' AS nodo, id, nombre, apellido, region, ciudad
    FROM    clientes
    WHERE   email LIKE 'escritura.%;'" 2>/dev/null

# Verificar que fraga fue eliminada de nodo04
mysql -h 192.168.56.104 \
-u spider_user \
-p'Spider_2025!' \
lab_bdd \
-e "SELECT 'nodo04' AS nodo, COUNT(*) AS filas_escritura
    FROM    clientes
    WHERE   email LIKE 'escritura.%;'" 2>/dev/null

# Verificar producto de prueba en nodo04
mysql -h 192.168.56.104 \
-u spider_user \
-p'Spider_2025!' \
lab_bdd \
-e "SELECT 'nodo04 v_productos_basico' AS nodo, id, sku, nombre
    FROM    v_productos_basico
    WHERE   id = 11;" 2>/dev/null
```

E.6 Crear vista de negocio y procedimiento almacenado en nodo06 (VM — nodo06)

 Bash

```
sudo mariadb lab_bdd << 'EOF'

-- =====
-- OBJETOS DE NEGOCIO: ENCAPSULACIÓN DE LA DISTRIBUCIÓN HÍBRIDA
-- =====
-- — Vista reporte_pedidos_detallado —————
-- Encapsula la fragmentación horizontal (clientes/pedidos/detalle)
-- y la fragmentación vertical (v_productos_basico).
-- El cliente consulta esta vista como si fuera una tabla local.
-- Usa v_productos_basico (no la VIEW productos) para evitar el
-- acceso innecesario a v_productos_detalle cuando solo se necesitan
-- datos operacionales.
CREATE OR REPLACE VIEW reporte_pedidos_detallado AS
SELECT
    c.region,
    CONCAT(c.nombre, ' ', c.apellido)      AS cliente,
    c.ciudad,
    c.email,
    p.id                                    AS pedido_id,
    p.fecha_pedido,
    p.estado,
    p.total                                AS total_pedido,
    vb.sku,
    vb.nombre                              AS producto,
    vb.categoria,
    vb.precio                              AS precio_catalogo,
    dp.cantidad,
    dp.precio_unitario,
    dp.subtotal
FROM    clientes          c
JOIN    pedidos           p  ON  p.cliente_id = c.id
JOIN    detalle_pedidos   dp ON dp.pedido_id  = p.id
JOIN    v_productos_basico vb ON vb.id         = dp.producto_id;
-- Verificar la vista
SELECT 'Vista reporte_pedidos_detallado — 5 filas de muestra'
AS verificacion;
SELECT region, cliente, ciudad, pedido_id, estado, producto,
categoria, subtotal
FROM    reporte_pedidos_detallado
ORDER  BY region, pedido_id
```

```

LIMIT 5;
-- Verificar que cubre todas las regiones
SELECT region, COUNT(*) AS lineas_de_detalle, ROUND(SUM(subtotal),2) AS total
FROM   reporte_pedidos_detallado
GROUP BY region
ORDER BY region;
-- — Procedimiento almacenado consulta_regional() —————
-- Parámetro NULL → todas las regiones (ambos shards horizontales)
-- Parámetro valor → poda Spider al shard correcto (una sola región)
-- Demuestra transparencia total: el llamador no sabe qué nodo se accede.
DROP PROCEDURE IF EXISTS consulta_regional;
DELIMITER //
CREATE PROCEDURE consulta_regional(
    IN p_region ENUM('norte','sur','este','oeste')
)
COMMENT 'Resumen de ventas por región. Pasar NULL para todas las regiones.'
BEGIN
    IF p_region IS NOT NULL THEN
        -- Con filtro: Spider poda horizontalmente al shard correcto.
        -- La consulta accede a UN SOLO nodo para los datos horizontales.
        SELECT c.region,
               COUNT(DISTINCT c.id) AS
clientes_activos,
               COUNT(DISTINCT p.id) AS
pedidos_totales,
               GROUP_CONCAT(DISTINCT vb.categoria
                             ORDER BY vb.categoria SEPARATOR ', ')
AS categorias,
               ROUND(SUM(dp.subtotal), 2) AS
facturacion,
               ROUND(SUM(dp.subtotal) / COUNT(DISTINCT p.id), 2)
AS ticket_promedio
        FROM   clientes          c
        JOIN   pedidos            p  ON  p.cliente_id = c.id
        JOIN   detalle_pedidos    dp ON dp.pedido_id  = p.id
        JOIN   v_productos_basico vb ON vb.id          = dp.producto_id
        WHERE  c.region = p_region
        GROUP BY c.region;
    ELSE
        -- Sin filtro: Spider accede a todos los shards horizontales.
        -- Equivale a H-6 del catálogo.
        SELECT c.region,
               COUNT(DISTINCT c.id) AS
clientes_activos,
               COUNT(DISTINCT p.id) AS

```

```

pedidos_totales,
        GROUP_CONCAT(DISTINCT vb.categoria
                        ORDER BY vb.categoria SEPARATOR ', ')
AS categorias,
        ROUND(SUM(dp.subtotal), 2) AS
facturacion,
        ROUND(SUM(dp.subtotal) / COUNT(DISTINCT p.id), 2)
AS ticket_promedio
FROM   clientes      c
JOIN   pedidos        p ON p.cliente_id = c.id
JOIN   detalle_pedidos dp ON dp.pedido_id = p.id
JOIN   v_productos_basico vb ON vb.id = dp.producto_id
GROUP BY c.region
ORDER BY facturacion DESC;
END IF;
END //
DELIMITER ;
-- Probar el procedimiento con distintos argumentos
SELECT '--- CALL consulta_regional(NULL) - todas las regiones ---' AS demo;
CALL consulta_regional(NULL);
SELECT '--- CALL consulta_regional(norte) - Spider poda a frag_A nodo04 ---'
AS demo;
CALL consulta_regional('norte');
SELECT '--- CALL consulta_regional(sur) - Spider poda a frag_B nodo05 ---'
AS demo;
CALL consulta_regional('sur');
SELECT '--- CALL consulta_regional(este) ---' AS demo;
CALL consulta_regional('este');
-- Confirmar que todos los objetos existen
SELECT TABLE_NAME AS objeto, TABLE_TYPE AS tipo, ENGINE
FROM   information_schema.TABLES
WHERE  TABLE_SCHEMA = 'lab_bdd'
ORDER BY TABLE_TYPE DESC, TABLE_NAME;
SELECT ROUTINE_NAME AS procedimiento, ROUTINE_TYPE, ROUTINE_COMMENT
FROM   information_schema.ROUTINES
WHERE  ROUTINE_SCHEMA = 'lab_bdd';
EOF

```

E.7 Demostración completa de transparencia de distribución (VM — nodo06)

 Bash

```
# Crear el usuario de aplicación final en nodo06
# Este usuario representa a un desarrollador o aplicación que:
# 1. Solo conoce la IP de nodo06 (192.168.56.106)
# 2. No sabe que existen nodo04 ni nodo05
# 3. Solo puede usar las tablas y vistas de lab_bdd en nodo06
sudo mariadb << 'EOF'
CREATE USER IF NOT EXISTS 'app_final'@'192.168.56.1'
  IDENTIFIED BY 'AppFinal_2025!';
GRANT SELECT, INSERT, UPDATE, DELETE ON lab_bdd.* TO
'app_final'@'192.168.56.1';
GRANT EXECUTE
      ON lab_bdd.* TO
'app_final'@'192.168.56.1';
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'app_final'@'192.168.56.1';
EOF
```

Conectarse como `app_final` desde el host Windows (simula la aplicación cliente):

 PowerShell

```
# Desde el host Windows — la aplicación solo conoce nodo06
mysql -h 192.168.56.106 -u app_final -p'AppFinal_2025!' lab_bdd
```

Dentro de la sesión como `app_final`, ejecutar las siguientes consultas de transparencia:

SQL

```
-- ¿Qué ve el usuario de aplicación?
SHOW TABLES;
-- Debe ver: clientes, pedidos, spider_detalle_nodo04, spider_detalle_nodo05,
--           v_productos_basico, v_productos_detalle y las VIEWS
-- Consulta 1: conteo global (accede a 2 shards transparentemente)
SELECT COUNT(*) AS total_clientes FROM clientes;
-- Consulta 2: buscar clientes del norte (Spider poda automáticamente)
SELECT id, nombre, apellido, ciudad
FROM   clientes
WHERE  region = 'norte'
ORDER BY id;
-- Consulta 3: ver catálogo de productos con ficha completa
-- (JOIN distribuido nodo04+nodo05 transparente)
SELECT id, sku, nombre, categoria, precio,
       LEFT(descripcion, 50) AS descripcion_preview,
       peso_kg
FROM   productos
ORDER BY categoria, nombre;
-- Consulta 4: reporte usando la vista de negocio
SELECT region,
       COUNT(*) AS lineas_detalle,
       ROUND(SUM(subtotal), 2) AS total
FROM   reporte_pedidos_detallado
GROUP BY region
ORDER BY total DESC;
-- Consulta 5: usar el procedimiento almacenado (todas las regiones)
CALL consulta_regional(NULL);
-- Consulta 6: procedimiento filtrado (solo este)
CALL consulta_regional('este');
-- Consulta 7: verificar que el usuario NO puede ver la
--             infraestructura interna
-- (no tiene acceso a mysql.servers ni a los shards directamente)
SELECT * FROM mysql.servers;
-- Error esperado: access denied (el usuario solo tiene permisos en lab_bdd)
```

Desde **nodo06** (sesión bddadmin), verificar que el usuario no puede ver los servidores:

 Bash

```
sudo mariadb << 'EOF'
-- Confirmar que app_final tiene los privilegios esperados y nada más
SHOW GRANTS FOR 'app_final'@'192.168.56.1';
-- Confirmar que la tabla mysql.servers está protegida
SELECT Server_name, Host, Db FROM mysql.servers;
-- Esto solo lo puede ver root/bddadmin, no app_final
EOF
```

E.8 Limpiar datos de prueba y apagar los nodos (VM — nodo06 y host)

 Bash

```
# En nodo06: eliminar datos de prueba de E.5
sudo mariadb lab_bdd << 'EOF'
-- Eliminar cliente de prueba que quedó en nodo05 (fragb)
DELETE FROM clientes WHERE email = 'escritura.fragb@lab.test';
-- Eliminar producto de prueba en nodo04
DELETE FROM v_productos_basico WHERE id = 11;
-- Verificar limpieza
SELECT 'clientes post-limpieza' AS verificacion, COUNT(*) AS total
FROM clientes;
SELECT 'v_productos_basico post-limpieza' AS verificacion, COUNT(*) AS total
FROM v_productos_basico;
-- Conteo final: debe coincidir exactamente con el estado inicial de la
Fase 14
SELECT 'clientes' AS tabla, COUNT(*) AS filas FROM clientes
UNION ALL
SELECT 'pedidos', COUNT(*) FROM pedidos
UNION ALL
SELECT 'detalle_pedidos', COUNT(*) FROM detalle_pedidos
UNION ALL
SELECT 'v_productos_basico', COUNT(*) FROM v_productos_basico
UNION ALL
SELECT 'v_productos_detalle', COUNT(*) FROM v_productos_detalle
UNION ALL
SELECT 'productos', COUNT(*) FROM productos;
EOF
```

Apagar cada nodo desde sus sesiones SSH activas:

 Bash

```
# En la sesión SSH de nodo06
sudo poweroff
# En la sesión SSH de nodo05
sudo poweroff
# En la sesión SSH de nodo04
sudo poweroff
```

Confirmar desde el host:

 PowerShell

```
Start-Sleep -Seconds 30
VBoxManage list runningvms
# La salida debe estar vacía
```

Tomar los snapshots en los seis nodos:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase15-completa" `
--description "Sin cambios en Fase 15. nodo01 apagado durante toda la fase.
Snapshot de hito."
VBoxManage snapshot "bdd-nodo02" take "fase15-completa" `
--description "Sin cambios en Fase 15. nodo02 apagado durante toda la fase.
Snapshot de hito."
VBoxManage snapshot "bdd-nodo03" take "fase15-completa" `
--description "Sin cambios en Fase 15. nodo03 apagado durante toda la fase.
Snapshot de hito."
VBoxManage snapshot "bdd-nodo04" take "fase15-completa" `
--description "SHARD-A: 4 indices nuevos (idx_region_cliente, idx_region_id,
idx_categoria, idx_precio). Datos verificados con catálogo H-1 a H-8. Escrituras
enrutadas desde nodo06. Datos limpios post-fase."
VBoxManage snapshot "bdd-nodo05" take "fase15-completa" `
--description "SHARD-B: 2 indices nuevos (idx_region_cliente, idx_region_id).
Datos verificados con catálogo híbrido. Datos limpios post-fase."
VBoxManage snapshot "bdd-nodo06" take "fase15-completa" `
--description "COORDINADOR: VIEW reporte_pedidos_detallado creada. PROCEDURE
consulta_regional() creado. Usuario app_final para transparencia. Catálogo H-1 a
H-8 verificado. EXPLAIN demuestra predicate pushdown."
```

Confirmar la lista de snapshots:

 PowerShell

```
VBoxManage snapshot "bdd-nodo04" list
VBoxManage snapshot "bdd-nodo05" list
VBoxManage snapshot "bdd-nodo06" list
```

nodo04 y nodo05 deben mostrar desde `fase05-completa` hasta `fase15-completa`. nodo06 muestra desde `fase14-completa` hasta `fase15-completa`.

F. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen **todos** los puntos siguientes:

1. `SHOW INDEX FROM pedidos` en nodo04 muestra los índices `idx_region_cliente` (columnas: `region, cliente_id`) e `idx_region_id` en `clientes` (columnas: `region, id`).
2. Los mismos índices `idx_region_cliente` e `idx_region_id` existen en nodo05.
3. `SHOW INDEX FROM v_productos_basico` en nodo04 muestra los índices `idx_categoria` y `idx_precio` además de los existentes de las fases anteriores.
4. La consulta H-1 (solo nodo04) devuelve **exclusivamente filas con regiones norte y este** y ninguna fila de `sur` u `oeste`.
5. La consulta H-2 devuelve **4 filas** (una por región) con datos de facturación no nulos.
6. La consulta H-3 devuelve **10 filas** con las columnas de `basico` y `detalle` combinadas (`nombre, descripcion, peso_kg`).
7. La consulta H-6 devuelve **exactamente 4 filas** con las cuatro regiones y datos de `categorias_compradas` y `facturacion_total` no nulos.
8. La consulta H-8 devuelve **2 filas** (`norte` y `este`) cuyos valores de `facturacion_total` coinciden exactamente con los de las mismas filas en H-6.
9. `EXPLAIN SELECT id, nombre FROM clientes WHERE region = 'norte'` muestra `partitions:`
`frag_A` únicamente (poda activa a nodo04).
10. `EXPLAIN SELECT id, nombre FROM clientes` sin filtro muestra `partitions:`
`frag_A, frag_B`.
11. `EXPLAIN SELECT COUNT(*) FROM detalle_pedidos` no muestra poda de partición (accede a ambas tablas subyacentes del UNION ALL).
12. `EXPLAIN SELECT id, nombre FROM v_productos_basico WHERE id = 1` muestra acceso a un solo nodo (no aplica partición, tabla Spider simple).
13. El INSERT con `region='norte'` desde nodo06 aparece en nodo04 al verificar directamente con `mysql -h 192.168.56.104 -u spider_user`.
14. El INSERT con `region='sur'` desde nodo06 aparece en nodo05 al verificar directamente con `mysql -h 192.168.56.105 -u spider_user`.
15. El UPDATE con predicado de región se aplica solo al shard correcto (verificado en el shard remoto).
16. La DELETE con predicado de región elimina la fila únicamente del shard correspondiente.
17. La VIEW `reporte_pedidos_detallado` existe en `information_schema.TABLES` con `TABLE_TYPE = 'VIEW'` y devuelve filas de las cuatro regiones.

18. `CALL consulta_regional(NULL)` devuelve 4 filas con datos de todas las regiones.
 19. `CALL consulta_regional('norte')` devuelve exactamente 1 fila (solo la región norte).
 20. `CALL consulta_regional('sur')` devuelve exactamente 1 fila (solo la región sur).
 21. El usuario `app_final` puede ejecutar `SELECT COUNT(*) FROM clientes` desde nodo06 y obtiene el resultado correcto (20 filas totales).
 22. El usuario `app_final` **no puede** ejecutar `SELECT * FROM mysql.servers` (error de permisos esperado).
 23. Después de la limpieza (E.8), `SELECT COUNT(*) FROM clientes` en nodo06 devuelve **20** y `SELECT COUNT(*) FROM v_productos_basico` devuelve **10**.
 24. Los snapshots `fase15-completa` existen en los **seis nodos** del laboratorio.
 25. El estudiante puede explicar de memoria la diferencia entre usar `detalle_pedidos` (VIEW) y `spider_detalle_nodo04` (tabla Spider directa), y cuándo conviene cada enfoque.
-

G. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>ALTER TABLE</code> <code>pedidos ADD</code> <code>INDEX IF NOT</code> <code>EXISTS</code> falla con error de sintaxis	La versión de MariaDB en uso no soporta <code>IF</code> <code>NOT EXISTS</code> en <code>ADD INDEX</code>	Usar <code>ALTER</code> <code>TABLE pedidos</code> <code>ADD INDEX</code> <code>idx_region_cli</code> <code>ente (region,</code> <code>cliente_id);</code> sin <code>IF NOT</code> <code>EXISTS</code> ; si el índice ya existe el motor devolverá un error 1061 inofensivo que se puede ignorar
H-8 no produce el mismo resultado que las filas norte+este de H-6	Los datos de <code>spider_detalle</code> <code>_nodo04</code> no coinciden con los de la VIEW <code>detalle_pedido</code> <code>s</code> filtrada por pedidos de frag_A	Verificar con <code>SELECT</code> <code>COUNT(*) FROM</code> <code>spider_detalle</code> <code>_nodo04</code> en nodo06 (debe coincidir con la suma de detalles de pedidos norte+este); si difiere, revisar la carga de la Fase 13 desde el snapshot
<code>EXPLAIN</code> no muestra la columna <code>partitions</code> para las tablas Spider	El cliente MySQL/MariaDB o la versión del motor omite la columna en la salida	Usar la sintaxis alternativa <code>EXPLAIN</code> <code>PARTITIONS</code> <code>SELECT ...</code> ; también se puede inferir la poda por los

		patrones de <code>rows</code> en la salida
INSERT desde nodo06 falla con <code>ERROR 1526: Table has no partition for value</code>	Se intentó insertar un valor de <code>region</code> que no existe en el dominio ENUM de las particiones Spider (por ejemplo, <code>'centro'</code>)	Verificar que el valor de <code>region</code> pertenece al ENUM <code>('norte', 'sur', 'este', 'oeste')</code> ; si el negocio requiere nuevos valores, ampliar el ENUM en los shards primero y luego agregar la partición en nodo06
UPDATE sin predicado de región falla con timeout	Spider busca la fila en ambos shards pero hay latencia alta o conexión lenta entre nodo06 y los shards	Aumentar <code>wait_timeout</code> en nodo06: <code>SET GLOBAL wait_timeout = 120;</code> ; asegurarse de que los tres nodos estén corriendo; verificar conectividad con <code>ping 192.168.56.10 4</code> desde nodo06
<code>CALL consulta_regional('norte')</code> devuelve 0 filas	Los datos de clientes con región='norte' fueron eliminados durante las pruebas o el procedimiento	Verificar con <code>SELECT COUNT(*) FROM clientes WHERE region='norte'</code> en nodo06; si es 0, revisar que las pruebas

	se creó antes de que los datos existieran	de limpieza no eliminaron datos válidos
La VIEW <code>reporte_pedido_s_detalhado</code> devuelve 0 filas	La VIEW usa <code>v_productos_ba_sico</code> que en nodo04 puede haberse vaciado si se limpió el id=11 también eliminó datos anteriores (no debería, DELETE WHERE id=11 es específico)	Verificar con <code>SELECT COUNT(*) FROM v_productos_ba_sico</code> en nodo06; si es menor de 10, restaurar nodo04 desde <code>fase14-completa</code>
El usuario <code>app_final</code> no puede conectarse desde el host Windows	El host de creación del usuario es <code>'192.168.56.1'</code> (IP del adaptador Host-Only del host Windows); si la IP del host no coincide, la conexión falla	Verificar la IP del adaptador VirtualBox Host-Only con <code>ipconfig</code> en Windows; ajustar el host en el <code>CREATE USER</code> al valor correcto
<code>SELECT * FROM mysql.servers</code> devuelve resultado vacío (no error) en nodo06	El usuario conectado tiene privilegios sobre <code>mysql.*</code> (no es <code>app_final</code>)	Confirmar que se está usando la sesión de <code>app_final</code> ejecutando <code>SELECT CURRENT_USER()</code>; si es <code>root</code> o <code>bddadmin</code>, la restricción no aplica por diseño
Los snapshots fallan con "VM is running"	Una de las VMs no terminó de apagarse antes del timeout	Esperar 30 segundos adicionales y verificar con <code>VBoxManage list</code>

		<code>runningvms</code> ; si persiste, usar <code>VBoxManage</code> <code>controlvm</code> <code>"bdd-nodo0X"</code> <code>acpipowerbutton</code> <code>n</code> y esperar
H-4 devuelve menos filas de las esperadas	<code>detalle_pedido</code> <code>s</code> VIEW puede no estar uniéndose correctamente con <code>pedidos</code> en el JOIN cuando hay filtro de región en <code>clientes</code> pero no en <code>pedidos</code>	Verificar que la consulta incluye <code>JOIN pedidos p</code> <code>ON</code> <code>p.cliente_id =</code> <code>c.id</code> sin filtro adicional en <code>pedidos</code> ; el filtro de región en <code>WHERE</code> <code>c.region IN</code> <code>(...)</code> debe ser suficiente para restringir los pedidos por transitividad del JOIN

H. Checklist de validación

- ☐ Los índices `idx_region_cliente` y `idx_region_id` existen en la tabla `pedidos` de nodo04 y nodo05.
- ☐ Los índices `idx_categoria` e `idx_precio` existen en `v_productos_basico` de nodo04.
- ☐ La consulta H-1 devuelve exclusivamente filas de las regiones `norte` y `este` .
- ☐ La consulta H-2 devuelve 4 filas (una por región) con datos de facturación.
- ☐ La consulta H-3 devuelve 10 filas con columnas de `v_productos_basico` y `v_productos_detalle` combinadas.
- ☐ La consulta H-6 devuelve exactamente 4 filas con `categorias_compradas` y `facturacion_total` no nulos.
- ☐ La consulta H-8 devuelve 2 filas cuyos valores coinciden con las filas de H-6 para `norte` y `este` .

- ☐ `EXPLAIN SELECT ... FROM clientes WHERE region = 'norte'` muestra poda a `frag_A` únicamente.
- ☐ `EXPLAIN SELECT ... FROM clientes` sin filtro muestra `frag_A, frag_B`.
- ☐ `EXPLAIN SELECT ... FROM detalle_pedidos` NO muestra poda de partición (VIEW UNION ALL).
- ☐ El INSERT con `region='norte'` desde nodo06 fue verificado en nodo04 directamente.
- ☐ El INSERT con `region='sur'` desde nodo06 fue verificado en nodo05 directamente.
- ☐ El UPDATE con predicado de región se aplicó solo al shard correcto.
- ☐ El DELETE con predicado de región eliminó la fila solo del shard correspondiente.
- ☐ La VIEW `reporte_pedidos_detallado` existe y devuelve filas de las 4 regiones.
- ☐ `CALL consulta_regional(NULL)` devuelve 4 filas.
- ☐ `CALL consulta_regional('norte')` devuelve 1 fila.
- ☐ `CALL consulta_regional('sur')` devuelve 1 fila.
- ☐ El usuario `app_final` puede ejecutar `SELECT COUNT(*) FROM clientes` en nodo06.
- ☐ El usuario `app_final` NO puede ejecutar `SELECT * FROM mysql.servers`.
- ☐ Después de la limpieza: `SELECT COUNT(*) FROM clientes` = 20 en nodo06.
- ☐ Después de la limpieza: `SELECT COUNT(*) FROM v_productos_basico` = 10 en nodo06.
- ☐ Los snapshots `fase15-completa` existen en los seis nodos.
- ☐ Puedo explicar la diferencia entre usar la VIEW `detalle_pedidos` y `spider_detalle_nodo04` directamente, y cuándo conviene cada opción.
- ☐ Puedo describir qué ocurre internamente en Spider ante un UPDATE sin predicado de región (fan-out).
- ☐ Puedo clasificar cualquier consulta dada en uno de los grados de localidad definidos en la Fase 15.

Preguntas teóricas para estudiantes

1. En el catálogo de consultas H-1 a H-8, la consulta H-4 accede a nodo04 (para datos horizontales `frag_A` y `v_productos_basico`) y a nodo05 (solo para `v_productos_detalle`), mientras que H-1 accede únicamente a nodo04. Ambas filtran por `region IN ('norte','este')`.

Explica con precisión qué diferencia en las tablas referenciadas provoca que H-4 requiera nodo05 y H-1 no. ¿Qué columna específica de qué tabla fuerza el acceso a nodo05 en

H-4?

¿Cómo podría reescribirse H-4 para eliminar ese acceso sin perder el resultado de negocio, si la descripción no fuera necesaria?

2. En la demostración de escrituras (E.5), el UPDATE sin predicado de región se calificó como *fan-out*. Describe en detalle el flujo de ejecución que Spider lleva a cabo internamente ante `UPDATE clientes SET telefono = '...' WHERE email = 'escritura.fragb@lab.test'`: ¿cuántas subconsultas genera?, ¿a qué nodos se envían?, ¿en qué orden?, ¿cómo determina Spider cuál shard contiene la fila antes de ejecutar la actualización? Propone una regla de diseño de aplicación que evite el fan-out en operaciones de escritura frecuentes.
3. La VIEW `detalle_pedidos` se implementa como `UNION ALL` de dos tablas Spider simples en lugar de una tabla Spider particionada por región (como `clientes` o `pedidos`). Ya se discutió que `detalle_pedidos` no tiene columna `region`. Un diseñador propone agregar una columna calculada `region_pedido` a `detalle_pedidos` en los shards (derivada del `pedido_id` vía la tabla `pedidos`) para poder usar `PARTITION BY LIST COLUMNS (region_pedido)` en nodo06. Evalúa esta propuesta: ¿qué DDL sería necesario en nodo04 y nodo05?, ¿qué mecanismo mantendría `region_pedido` sincronizado con `pedidos.region` sin FK de motor?, ¿qué mejora de rendimiento produciría en la consulta H-5 específicamente?, y ¿qué riesgo de consistencia introduce?
4. La VIEW `reporte_pedidos_detallado` usa `v_productos_basico` directamente en lugar de la VIEW `productos` (JOIN basico+detalle). Esta decisión optimiza el acceso pero limita las columnas disponibles en el reporte. Describe el proceso de decisión técnica completo que debería seguir un DBA para determinar si una vista de este tipo debe usar `v_productos_basico` o `productos`: ¿qué métricas de frecuencia de acceso a columnas consultaría?, ¿qué impacto tiene en el tráfico de red entre coordinador y nodo05?, ¿cómo afecta al tiempo de respuesta si el 80% de las consultas a la vista no necesitan descripción ni ficha técnica?
5. El procedimiento `consulta_regional('norte')` logra Grado 0 de localidad para los datos horizontales (accede solo a nodo04 para clientes, pedidos y detalles) pero `v_productos_basico` también está en nodo04, de modo que la consulta realmente accede a un solo nodo físico. Sin embargo, si el procedimiento necesitara incluir `peso_kg` del producto, forzaría el acceso

a `v_productos_detalle` en nodo05. Clasifica las siguientes variantes del procedimiento según el grado de localidad que producen cuando `p_region = 'norte'`:

- (a) agregar `vb.stock` al SELECT sin cambiar el JOIN;
- (b) agregar `vd.peso_kg` al SELECT con JOIN adicional a `v_productos_detalle`;
- © cambiar el WHERE a `WHERE c.region IN ('norte','sur')`;
- (d) eliminar el WHERE completamente.

Ejercicios prácticos

1. Catálogo extendido: tres consultas propias con análisis de localidad.

Diseñar, escribir y ejecutar tres consultas adicionales sobre el esquema distribuido de nodo06 que no estén en el catálogo H-1 a H-8. Para cada una: (a) especificar qué fragmentos accede Spider (indicando nombre de tabla y nodo físico), (b) asignar el grado de localidad según la clasificación de la Fase 15, © ejecutar `EXPLAIN` y registrar la columna `partitions` y `rows` en la salida, y (d) proponer una versión optimizada que reduzca el grado de localidad en al menos un nivel, manteniendo la corrección del resultado. Documentar la salida de `EXPLAIN` para la versión original y la optimizada.

2. Benchmark comparativo: VIEW detalle_pedidos vs. tablas Spider directas.

Diseñar un experimento que mida y compare el rendimiento de las dos estrategias de acceso a `detalle_pedidos` desde nodo06:

- (a) Usando la VIEW (`JOIN detalle_pedidos ON ...`) con filtro de región en `clientes` .
- (b) Usando la tabla Spider directa (`JOIN spider_detalle_nodo04 ON ...`) con el mismo filtro de región.

Para medir el rendimiento, usar `FLUSH STATUS; [CONSULTA]; SHOW STATUS LIKE 'Handler_read%';` antes y después de cada consulta, registrando el valor de `Handler_read_rnd_next` . Repetir cada consulta 5 veces, calcular el promedio, comparar y concluir: ¿en qué porcentaje reduce el acceso directo a la tabla Spider el valor de `Handler_read_rnd_next` comparado con la VIEW? ¿Es la diferencia proporcional al número de filas en el shard que no se necesitan?

3. Ampliación del procedimiento: parámetro de categoría de producto.

Modificar el procedimiento `consulta_regional()` para aceptar un segundo parámetro `IN p_categoria VARCHAR(100)` . Cuando `p_categoria` no es NULL, la consulta debe filtrar también por la categoría del producto en `v_productos_basico.categoria` . Implementar

las cuatro combinaciones: (NULL, NULL), (region, NULL), (NULL, categoria) y (region, categoria). Para cada caso, ejecutar `EXPLAIN` e identificar las particiones accedidas.

Documentar si el filtro por categoría cambia el grado de localidad de la consulta y explicar por qué sí o no lo hace (pista: categoría está en `v_productos_basico`, que siempre está en nodo04; ¿cómo afecta esto al grado de localidad cuando la región es NULL?).

Reto adicional para alumnos avanzados

Investigar e implementar un sistema de **caché de resultados híbrida** en nodo06 usando una tabla InnoDB local que almacene el resultado de la consulta H-6 (resumen por región) con una marca de tiempo de validez:

SQL

```
CREATE TABLE cache_resumen_regional (  
  region      ENUM('norte','sur','este','oeste') PRIMARY KEY,  
  clientes    INT,  
  pedidos     INT,  
  categorias  TEXT,  
  facturacion DECIMAL(12,2),  
  generado_en DATETIME DEFAULT CURRENT_TIMESTAMP,  
  valido_hasta DATETIME  
) ENGINE = InnoDB;
```

El reto consiste en: (1) crear un procedimiento `refrescar_cache(p_ttl_minutos INT)` que ejecute H-6 y cargue sus resultados en `cache_resumen_regional` con un TTL configurable en minutos; (2) crear un procedimiento `consulta_con_cache(p_region ...)` que devuelva el resultado de `cache_resumen_regional` si es válido (no ha expirado) o ejecute H-6 y refresque la caché si el TTL venció; (3) demostrar el comportamiento midiendo el tiempo de ejecución con `TIMEDIFF(NOW(), t_inicio)` para la primera llamada (caché vacía, H-6 real) y la segunda (caché válida, solo lectura local). Documentar el trade-off: ¿qué consistencia se pierde al usar caché? ¿Qué tipo de inconsistencia es posible si se insertan pedidos en los shards durante el TTL de la caché? ¿Cómo podría el sistema invalidar la caché ante una escritura, dado que Spider no tiene trigger inter-nodo?

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Índices de optimización en shards	10%	Los cuatro índices existen en nodo04 y los dos en nodo05; el estudiante puede justificar el patrón de consulta que cubre cada índice y explicar por qué mejora el rendimiento de las subconsultas Spider
Catálogo de consultas híbridas	25%	Las ocho consultas H-1 a H-8 devuelven resultados correctos y no vacíos; el estudiante puede describir para cada una los fragmentos accedidos y asignar el grado de localidad correcto sin consultar el documento
Análisis EXPLAIN y optimización	20%	El estudiante demuestra con evidencia de EXPLAIN que el predicate pushdown actúa en tablas Spider particionadas; puede explicar

		por qué la VIEW <code>detalle_pedido</code> <code>s</code> no se beneficia del mismo mecanismo y propone la alternativa correcta
Operaciones de escritura	15%	Los INSERT se verificaron en los shards correctos; el estudiante puede describir el flujo de fan-out en UPDATE sin predicado de región y propone una regla de diseño para evitarlo
Objetos de negocio (VIEW y SP)	20%	<code>reporte_pedido</code> <code>s_detallado</code> y <code>consulta_regional()</code> existen y producen resultados correctos; el estudiante puede explicar qué fragmentos accede cada objeto y por qué se eligió <code>v_productos_basico</code> en lugar de <code>productos</code>
Demostración de transparencia	10%	El usuario <code>app_final</code> ejecuta consultas correctas sin conocer la

		distribución; el estudiante articula los tres tipos de transparencia implementados (fragmentación, ubicación, replicación) con ejemplos concretos del laboratorio
--	--	---

I. Preparación para la siguiente fase

La **Fase 16: Consultas Distribuidas** requerirá:

- Los snapshots `fase15-completa` en los seis nodos activos del laboratorio (completados en esta fase).
- `bdd-nodo06` con Spider activo, los objetos creados en esta fase (`reporte_pedidos_detallado`, `consulta_regional()`) y el usuario `app_final` operativo.
- La comprensión de los grados de localidad establecida en esta fase: la Fase 16 presentará estrategias formales de descomposición de consultas globales en subconsultas, la semijoin distribuida como técnica de optimización, y el impacto del volumen de datos intermedios transferidos entre coordinador y shards.
- Los índices creados en nodo04 y nodo05 en esta fase, que serán la base sobre la que la Fase 16 medirá el impacto de la optimización de consultas distribuidas.

En la Fase 16 se formalizará el proceso de **descomposición de consultas distribuidas**:

dado un SQL global emitido en nodo06, se identificarán las subconsultas elementales que Spider

genera internamente, se calcularán los volúmenes de datos intermedios transferidos por la red y se aplicarán técnicas de semijoin y filtrado temprano para reducir ese volumen. La Fase 16 también introducirá el concepto de **join distribuido con poda por índice**: cómo una condición sobre un fragmento horizontal puede reducir el número de filas que Spider necesita recuperar del fragmento vertical, y viceversa.

